03

Source of truth

Where each piece of data is stored, and what happens when copies disagree

A value on screen has usually been copied several times on its way there: a database row, a server response, a client-side cache, a component's state, a form's initial value. Each copy can age or diverge, and most bugs where the screen shows the wrong thing trace to two copies with no rule for which one is right. This chapter is the set of distinctions that make those bugs diagnosable: stored against derived, the server's copy against the client's, and what happens when two operations run at once.

Assumes chapter 1's two sides; chapter 2 covers the stored side's shape.

PART

The model

Stored and derived values

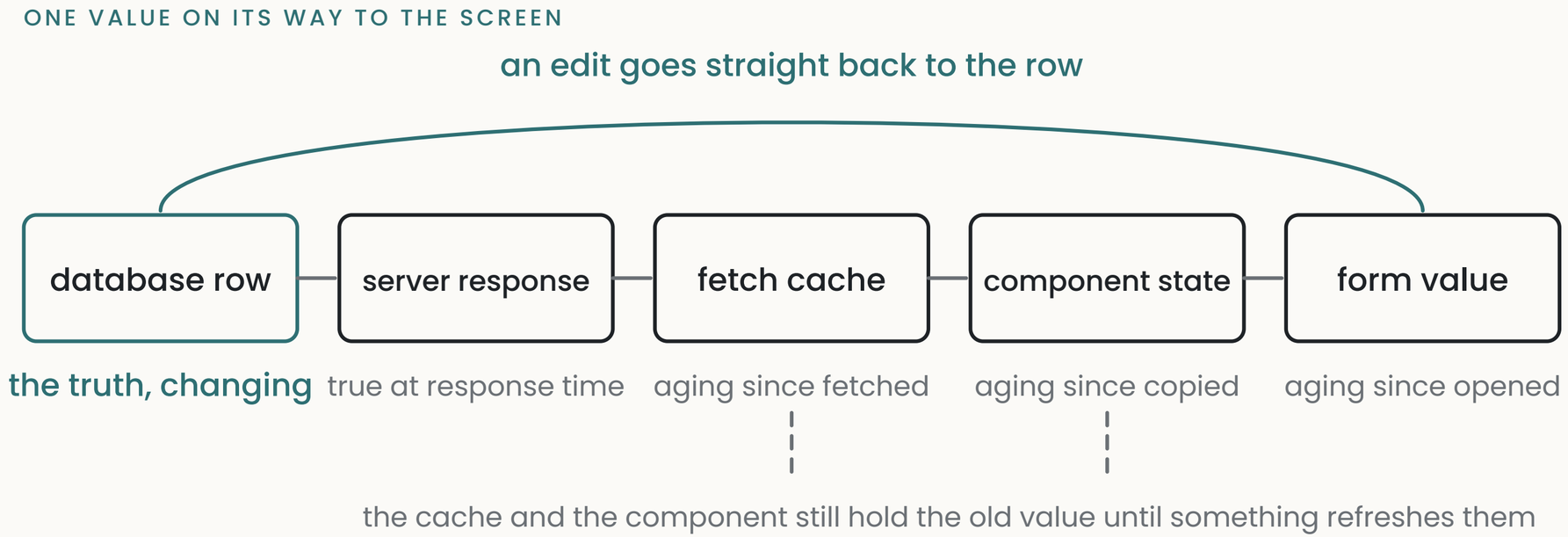
Every value is either recorded directly or computable from other recorded values. An order's line items are stored; its total is derivable. The design rule is that each fact gets one authoritative home, and everything derivable is computed at read time — because a stored copy of a derivable value is a second home, and two homes need a synchronization mechanism, which is code that can fail halfway. Storing a derived value is sometimes justified by cost (recomputing is expensive), and then the synchronization mechanism gets named and checked rather than assumed.

The server's copy and the client's copies

When the browser fetches data, it receives a copy that was correct at the moment of the response. The database keeps changing after that moment, and nothing notifies the copy.

Fetched data is therefore a cache: a copy kept for speed, aging from the moment it arrives.

The copies multiply quietly; a typical path holds five: the database row, the server's response, the client's fetch cache, a component's state (a component is one piece of the interface — a form, a list — with its own memory), and a form's initial value. Duplication starts where fetched data gets treated as local state — copied out of the cache into a component, after which the two age independently and edits land in one but not the other.



Staleness

Every cache implies an answer to one question: when is this copy refreshed? Common answers are on a timer, when the user returns to the window, and after any write that would change it. “Never” is also an answer, and it’s the right one only for data that never changes — for anything else it’s a bug that presents as users seeing old data until they reload.

Two operations at once

Requests take time, so two can be in flight together, and responses can return in a different order than the requests were sent. A search box that fires a request per keystroke can receive the results for “a” after the results for “ab”, and if the code applies whichever response arrived last, the screen shows results for a query the user is no longer making.

This is a **race condition**: an outcome that depends on timing rather than on logic. The general guard is to make the code state which operation's result it's waiting for and ignore the superseded ones.

Operations safe to repeat

An operation is **idempotent** if running it twice has the same effect as running it once. Setting a title is idempotent; appending a comment isn't. The property matters here because repeats happen without anyone choosing them — double-clicks, retries after a timeout, a refresh mid-submit — so any operation that creates or accumulates needs either a guard against repeats or an **idempotency key**: an identifier supplied with the operation so the receiver can recognize the second attempt as a duplicate. Chapter 9 applies this to calls that leave your system.

The URL as a home for state

Which screen the user is on, what's selected, which filters are active — this state describes where the user is, and when it lives only in component memory, three things break at once: refresh loses it, the back button misbehaves, and the view can't be shared as a link. State of this kind belongs in the URL, which makes the browser's own history mechanism maintain it.

PART

What goes wrong

1 One value, three copies

the same data held in several client-side homes that age independently.

ASK “List every place this data exists on the client, and for each: what writes it and what refreshes it.”

CHECK change the value in one place and look at the others.

TELL *server data appearing in a fetch cache, in component state, and in a form’s initial value.*

2 Optimistic update with no rollback

the screen assumes the write will succeed and has no plan for when it doesn’t.

ASK “For each place the interface updates before the server confirms, state what happens when the server rejects the write, and what the user sees.”

CHECK block the request in devtools (D4 shows how) and watch the screen.

TELL *a local update applied before the request is sent, with no handling of a rejected response.*

3 Failure after the user has typed

a rejected submission that discards the input.

ASK “Walk through a submission that fails after thirty seconds. What does the user see at each moment, and is their input still there at the end?”

CHECK block the request in devtools and submit a filled form.

TELL *no submission state, and no branch for a failed response.*

4 State that belongs in the URL

a view that can’t survive a refresh or be linked to.

ASK “List every piece of state that describes where the user is or what they’re looking at, and whether each survives a refresh. Say no explicitly.”

CHECK set up a filtered view, refresh, then press back.

TELL *filters, selections, or the current item held only in component memory.*

THE DIRECT TEST

Open your app in two windows side by side. Change something in one and watch whether, and when, the other notices. Then block the network in one window's devtools and try a write — D4 below has your assistant show you how, and chapter 0's D2 is the shorter version. It takes about ten minutes, and the results are your app's actual answers to the staleness question and the rollback question, whatever the code was intended to do.

PART

Going deeper

These prompts are for your assistant, and each comes with a note on what a good response looks like.

D1 · Inventory the copies

AUDIT

Pick the most important piece of data in my app: [name it]. List every place a copy of it exists between the database and the screen — server responses, caches, stores, component state, form values. For each copy: what writes it, what refreshes it, and what the user sees if it's stale. Full list, as a table.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response has more rows than you expected and an honest answer in every "what refreshes it" cell — "nothing" is a finding, and so is two copies refreshed by different events.

D2 · Trace one value there and back

GROUNDED EXPLANATION

Trace [the same piece of data] from its database row to the pixels, and then back through an edit: every copy made, every transformation applied, and the moment the database is updated. Point at the actual files at each step.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is a sequence you could follow with the files open. The finding to watch for is a copy on the way out that the edit path doesn't update.

D3 · Walk the race

WALK THE FAILURE PATH

Walk me through what happens when the same record is edited twice at nearly the same time — a double-click on save, or two tabs open on the same item. Step by step: both requests, the order the database sees them, what the final stored state is, and whether anything tells the user whose edit lost.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response reaches a definite final state and names which write wins. “That can’t happen” is a claim to test — the double-click version takes seconds to try.

D4 · Force the failure

BUILD A TOY

Help me use devtools to test my own app’s failure behaviour: block the network mid-write and watch what the screen claims; then submit a form with the request blocked and see whether my input survives; then throttle the connection and watch which screens go stale.

WHAT A GOOD RESPONSE LOOKS LIKE

About twenty minutes of work turns this chapter’s entries into things you’ve seen on your own screens. Each surprise is an entry’s check that just failed.

WHERE THIS CONNECTS

Chapter 2 · Data modelling covers the stored side — its derived-columns entry and this chapter’s stored-against-derived section are the same rule on either side of the database boundary. **Chapter 9 · Third-party integrations** applies idempotency to calls that leave your app. **Chapter 4 · The request lifecycle** names the path these copies travel.